Assignment 1: GRASPing Expensive Insertion

1CM260 Large-scale Optimization for Transport & Mobility

Yusuf Berkan Gokce

Student ID: 2372711

Quarter 1



Eindhoven, September 17, 2026

1 Introduction

The Traveling Salesman Problem (TSP) is a classical combinatorial optimization problem in which the objective is to find the shortest tour visiting every city exactly once. Since the number of possible tours grows rapidly with problem size, heuristic methods provide a practical approach for obtaining good solutions efficiently.

This assignment investigates several TSP heuristics on ten benchmark instances. Nearest Neighbour and Expensive Insertion are used as constructive heuristics, after which Expensive Insertion is extended using GRASP. The resulting tours are further improved using 2-opt local search, including a comparison between best-improvement and AI-generated first-improvement strategies. Finally, GRASPed insertion is combined with 2-opt local search.

The results show that Nearest Neighbour is sensitive to its starting city and that GRASP can improve upon deterministic Expensive Insertion. Both 2-opt implementations produced 2-optimal solutions, although they often reached different local optima. Combining GRASPed insertion with 2-opt further improved the tour quality compared with the GRASP construction alone.

2 Methodology

2.1 Part 1: Selection of TSP Instances

Ten Euclidean TSP benchmark instances were selected as specified in the assignment: five small (berlin52, eil51, eil76, kroA100, kroA200), three medium (a280, ali535, fl417), and two large (rl1304, fl1577). The different instance sizes allow the heuristics to be evaluated across varying problem sizes, while using the same instances throughout ensures consistent comparisons between methods. A fixed NumPy random seed of 42 was used for randomized experiments to ensure reproducibility.

2.2 Part 2: Nearest Neighbour Heuristic

The nearest neighbour (NN) heuristic was evaluated for every possible starting city of each of the ten selected TSP instances. For a given starting city, the heuristic repeatedly visits the closest city that has not yet been visited until a complete tour is constructed. The total cost of each resulting tour was then computed.

To investigate the sensitivity of the NN heuristic to its starting position, the obtained tour costs were plotted against their corresponding starting cities for each instance. This allows the variation in solution quality caused solely by changing the initial city to be visualized. The starting city producing the minimum tour cost was also recorded for each instance.

2.3 Part 3: Expensive Insertion Heuristic

The expensive insertion heuristic was implemented in the method `getTour_ExpensiveInsertion`. Starting from a given city, the city farthest from the starting city is first selected to form an initial two-city tour. The remaining cities are then inserted iteratively until all cities are included. At each iteration, two decisions are made. First, for every city c not yet in the tour T, its minimum distance to any city already in the tour is computed. The city with the largest of these minimum distances is selected:

$$c^* = \arg\max_{c \notin T} \left(\min_{j \in T} d(c, j) \right).$$

Thus, the selected city is the unvisited city that is farthest from the current partial tour. Second, the selected city c^* is inserted into the position that causes the smallest increase in tour cost. For every edge (i, j) in the current tour, the insertion cost is computed as

$$\Delta_{ij} = d(i, c^*) + d(c^*, j) - d(i, j).$$

The edge with the smallest value of Δ_{ij} is replaced by (i, c^*) and (c^*, j). These two steps are repeated until every city has been inserted into the tour.

2.4 Part 4: GRASPed Expensive Insertion

The expensive insertion heuristic contains two decisions that can be randomized. The first is the selection of the next city to be added to the partial tour. In the deterministic version, the city with the largest minimum distance to the current tour is always selected. The second is the selection of the insertion position. Once a city has been selected, the deterministic heuristic inserts it at the position that results in the smallest increase in tour cost.

In the implemented GRASP variant, the city-selection step was randomized, while the insertion-position step was kept greedy. For every unvisited city c, a candidate score was calculated as

$$g(c) = \min_{j \in T} d(c, j),$$

where T denotes the set of cities currently in the tour. A larger value of $g(c)$ therefore indicates that city c is farther from its nearest city in the current tour. The best candidate score is given by

$$g_{\text{best}} = \max_{c \in U} g(c),$$

where U is the set of unvisited cities.

Instead of always selecting the city with g_{best}, a Restricted Candidate List (RCL) is constructed according to

$$\text{RCL} = \{c \in U : g(c) \geq (1 - \alpha) g_{\text{best}}\}.$$

A city is then selected uniformly at random from the RCL. The parameter α controls the amount of randomization. When $\alpha = 0$, only candidates with the best score can be selected, reproducing the greedy city-selection rule, apart from possible ties. Increasing α allows a larger set of near-best candidates to enter the RCL.

After selecting a city from the RCL, its insertion position is determined greedily as in the original expensive insertion heuristic. For every edge (i, j) in the current tour, the insertion cost

$$\Delta = d(i, c) + d(c, j) - d(i, j)$$

is evaluated, and the position with the smallest Δ is selected. Thus, randomization is used to diversify which cities are selected during construction, while the insertion of each selected city remains greedy.

Since a single GRASP construction is randomized, the complete construction is repeated for a predefined number of iterations. Each iteration constructs a complete feasible tour independently using the same starting city and α value, and its total cost is evaluated. The lowest-cost tour obtained over all iterations is returned as the final GRASP solution. In the experiment on `berlin52`, $\alpha = 0.10$ and 100 GRASP iterations were used.

2.5 Part 5: 2-opt Local Search

Two methods were implemented for 2-opt local search: `isTwoOpt`, which checks whether a tour is 2-optimal, and `makeTwoOpt`, which repeatedly applies improving 2-exchanges until a 2-optimal solution is reached.

A 2-exchange considers two non-adjacent edges (a, b) and (c, d) in the current tour. The cost of these two edges before the exchange is

$$C_{\text{old}} = d(a, b) + d(c, d),$$

while reconnecting them as (a, c) and (b, d) gives

$$C_{\text{new}} = d(a, c) + d(b, d).$$

Therefore, an improving 2-exchange exists whenever

$$C_{\text{new}} < C_{\text{old}}.$$

The `isTwoOpt` method systematically scans all valid pairs of non-adjacent edges. Candidate pairs are considered according to their positions in the tour: the first edge is scanned from the beginning to the end of the tour, while the second edge is scanned starting two positions after the first. This avoids pairs of consecutive edges that share a city. Since the tour is closed, the pair consisting of the first and last edges is also excluded. If any improving exchange is encountered, the method immediately returns `False`. If the complete set of candidate pairs is scanned without finding an improvement, the tour is declared 2-optimal and `True` is returned.

The `makeTwoOpt` method uses a best-improvement strategy. During each local-search iteration, all valid pairs of non-adjacent edges are scanned in the same order as in `isTwoOpt`. For each candidate exchange, the change in cost is calculated as

$$\Delta = [d(a, c) + d(b, d)] - [d(a, b) + d(c, d)].$$

A negative value of Δ represents an improvement. Rather than applying the first improving exchange encountered, the complete neighbourhood is searched and the exchange with the most negative Δ is retained. This best-improvement strategy was selected to apply the largest available reduction in tour cost at each iteration, although it requires evaluating the complete 2-opt neighbourhood before each exchange.

Once the best pair of edges has been identified, the section of the tour between the two selected edges is reversed. This replaces (a, b) and (c, d) with (a, c) and (b, d) while maintaining a single feasible tour. The procedure is repeated until `isTwoOpt` finds no further improving 2-exchange.

To validate both methods, the tours previously generated using Nearest Neighbour (NN), Expensive Insertion (EI), and GRASPed Expensive Insertion were used as input. First, `isTwoOpt` was applied to determine whether each constructed tour was already 2-optimal. Subsequently, `makeTwoOpt` was applied to each tour and the resulting solutions were checked again using `isTwoOpt`. Finally, `isFeasible` was applied to the improved tours to verify that the 2-exchanges preserved feasibility.

2.6 Part 6: Comparison with AI-generated 2-opt

Following the assignment instructions, the prescribed prompt from Appendix A was submitted to ChatGPT in a new conversation without providing any additional context. The generated method was added to the TSP class as `makeTwoOpt_AI`. The complete interaction with the AI assistant, including the original prompt and generated code, is documented in Appendix A.

Although both implementations apply 2-exchanges until no improving exchange remains, they differ in their local-search strategy. The implementation developed in Part 5, `makeTwoOpt`, uses a *best-improvement* strategy. In each iteration, all valid pairs of non-adjacent edges are evaluated and the exchange producing the largest reduction in tour cost is selected. Only after the complete neighbourhood has been scanned is the best exchange applied.

In contrast, the AI-generated `makeTwoOpt_AI` method uses a *first-improvement* strategy. Candidate edge pairs are scanned in order, and as soon as an improving exchange is encountered, it is applied immediately. The search then restarts from the beginning of the neighbourhood. Consequently, the AI implementation can avoid scanning the complete neighbourhood whenever an improvement is found, whereas the best-improvement implementation performs a complete neighbourhood scan before each exchange.

For a fair comparison, both implementations were evaluated on all ten selected TSP instances. For each instance, an identical initial tour was generated using the Nearest Neighbour heuristic with the same starting city. A copy of this same initial tour was then supplied separately to `makeTwoOpt` and `makeTwoOpt_AI`, ensuring that differences in the resulting solutions were caused by the local-search strategies rather than different initial solutions. The random seed was also kept identical throughout the experimental setup, although the NN and 2-opt procedures themselves are deterministic once the starting city is fixed.

Three aspects were compared. First, the final tour costs were recorded to compare solution quality. Second, `isTwoOpt` was applied to both outputs to verify that each method terminated at a 2-optimal solution. Finally, computation time was measured using `time.perf_counter()` to compare the computational performance of the two implementations.

2.7 Part 7: GRASPed Insertion with 2-opt Local Search

The GRASPed expensive insertion heuristic from Part 4 was combined with the 2-opt local search developed in Part 5. The implementation reuses the existing methods `getTour_GRASPedInsertion` and `makeTwoOpt` rather than implementing the construction and local-search procedures again.

The combined method consists of two levels of iteration. An outer loop is performed for 500 iterations. Within each outer iteration, `getTour_GRASPedInsertion` is called with 10 internal GRASP iterations. Consequently, 10 randomized GRASP tours are constructed and the best of these is returned by the GRASP method. This tour represents the constructive solution for the current outer iteration. Its objective value is stored as the cost before local search.

The selected GRASP tour is subsequently passed to `makeTwoOpt`, which applies the best-improvement 2-opt procedure described in Part 5 until no improving 2-exchange remains. The resulting objective value is stored as the cost after local search. The best locally improved tour encountered over all outer iterations is retained as the final solution. Therefore, with 500 outer iterations and 10 GRASP constructions per outer iteration, a total of 5000 randomized constructions are generated, while 500 selected tours are subjected to 2-opt local search.

For consistency with the previous experiments, the method was evaluated on `berlin52` using starting city 39 and $\alpha = 0.10$. A fixed random seed was used to make the randomized experiment reproducible.

The costs before and after local search were recorded for every outer iteration and plotted against each other, with the constructive GRASP cost on the x-axis and the corresponding cost after 2-opt on the y-axis. A $y = x$ reference line was included to indicate cases in which local search produced no improvement.

3 Results

3.1 Part 2: Nearest Neighbour Heuristic

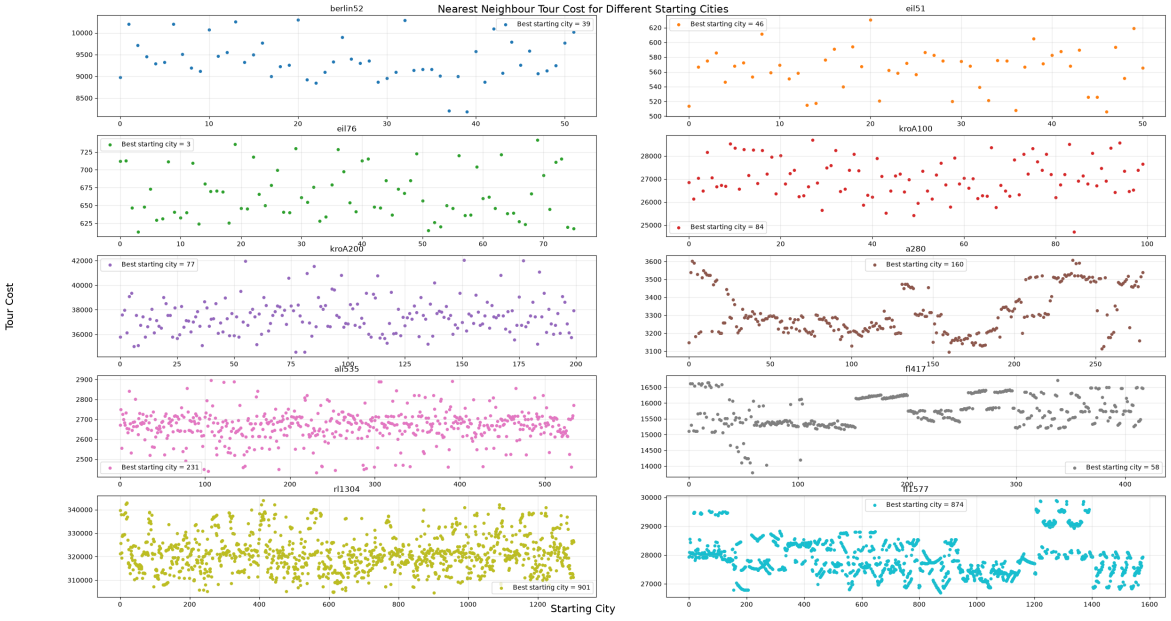


Figure 1: Nearest neighbour tour costs obtained from every possible starting city for the ten selected TSP instances.

Figure 1 shows the tour costs obtained by applying the nearest neighbour heuristic from every possible starting city for each of the ten selected instances. The results show that the starting city influences the resulting solution quality, although the magnitude and structure of this variation differ considerably between instances.

For each instance, the starting city that produced the lowest tour cost was also identified and is indicated in the corresponding subplot in Figure 1. In particular, for the `berlin52` instance, which was used as the main test instance for the developed methods in the subsequent parts, city 39 produced the best nearest neighbour solution and was therefore used as the starting city in these tests.

Furthermore, there is no common starting-city index that consistently produces the best solution across the instances. The best starting city changes from one instance to another, indicating that a good starting position is instance-dependent. This sensitivity is a consequence of the greedy nature of the NN heuristic: the first city affects the first selected neighbour, which in turn influences subsequent decisions and can lead to a different final tour.

3.2 Part 3: Expensive Insertion Heuristic

The implementation was tested on the `berlin52` instance using city 39 as the starting city. The expensive insertion heuristic produced a feasible tour with a total cost of 8451.88. For comparison, the nearest neighbour heuristic from the same starting city produced a tour cost of 8182.19. Although expensive insertion resulted in a higher cost for this particular test, the purpose of this experiment was primarily to verify the correctness of the implementation. The generated solution contained every city exactly once and was therefore confirmed to be feasible.

3.3 Part 4: GRASPed Expensive Insertion

The GRASPed expensive insertion heuristic was tested on the `berlin52` instance using starting city 39, $\alpha = 0.10$, and 100 construction iterations. The best tour found had a cost of 7996.15 and was verified to be feasible. For comparison, the deterministic expensive insertion heuristic from Part 3, using the same starting city, produced a tour cost of 8451.88. The GRASP procedure therefore found a lower-cost

tour in this experiment. This shows the benefit of introducing diversification into the city-selection step: different choices from the RCL generate different construction paths, allowing the algorithm to explore solutions that cannot be reached by the deterministic construction from the same starting city. The use of multiple iterations then allows the best of these randomized constructions to be retained.

3.4 Part 5: 2-opt Local Search

Table 1: Validation of the 2-opt implementation on the `berlin52` tours.

Initial heuristic	Initially 2-optimal	After 2-opt	Feasible
Nearest Neighbour	False	True	True
Expensive Insertion	False	True	True
GRASPed Insertion	True	True	True

Table 1 summarizes the validation of the 2-opt implementation. The NN and Expensive Insertion tours were not initially 2-optimal, whereas the GRASPed Insertion tour was already 2-optimal. After applying `makeTwoOpt`, all three tours were accepted by `isTwoOpt`. Furthermore, all resulting tours remained feasible, confirming that the implemented exchanges preserve the validity of the TSP tour.

3.5 Part 6: Comparison with AI-generated 2-opt

Table 2 presents the comparison between the best-improvement implementation developed in Part 5 and the AI-generated first-improvement implementation. Both algorithms were started from identical NN tours for each instance.

Table 2: Comparison of the developed and AI-generated 2-opt implementations.

Instance	Initial	Best-imp.	First-imp.	Time Best (s)	Time First (s)	2-optimal
berlin52	8182.19	7713.03	7669.62	0.0064	0.0045	Both
eil51	570.92	459.86	467.44	0.0095	0.0047	Both
eil76	641.63	565.66	574.68	0.0273	0.0067	Both
kroA100	26310.51	21395.37	22089.38	0.0665	0.0287	Both
kroA200	36658.69	30189.12	30683.44	0.5859	0.3265	Both
a280	3322.51	2761.93	2722.18	0.9365	1.0632	Both
ali535	2738.06	2162.50	2190.98	9.1354	6.3698	Both
fl417	15339.44	12340.81	12252.86	4.5081	5.0806	Both
rl1304	324666.10	273230.93	273831.53	80.7731	81.7411	Both
fl1577	29439.94	23070.92	23166.53	155.4874	171.4071	Both

Both implementations successfully produced 2-optimal solutions for all ten instances, as verified using `isTwoOpt`. This is expected because the two methods follow different trajectories through the 2-opt neighbourhood. The best-improvement method always applies the largest improvement available in the current neighbourhood, whereas the first-improvement method accepts the first improving move encountered. Since 2-opt can contain multiple local optima, these different move sequences can lead to different final tours even when starting from exactly the same initial solution.

Neither strategy consistently produced the lower final tour cost across all instances. The AI-generated first-improvement method obtained a lower cost for `berlin52`, `a280`, and `fl417`, while the developed best-improvement method obtained a lower cost for the remaining seven instances. For example, on `berlin52`, first-improvement reached a cost of 7669.62 compared with 7713.03 for best-improvement. Conversely, on `kroA100`, best-improvement obtained 21395.37 compared with 22089.38 for first-improvement. Thus, selecting the largest immediate improvement does not guarantee the best final local optimum, although it resulted in the lower final cost on more of the tested instances.

The computation times also show that first-improvement is not uniformly faster. For several small and medium instances, including `eil51`, `eil76`, `kroA100`, and `kroA200`, the AI-generated first-improvement method required less computation time. This is consistent with its ability to stop scanning the neighbourhood immediately after finding an improving move. For example, first-improvement was slower on `a280`, `fl417`, `rl1304`, and `fl1577`. Although an individual first-improvement iteration can terminate its neighbourhood scan earlier, the method may require a different number of accepted exchanges and repeated scans before reaching a local optimum.

Overall, both implementations correctly reached 2-optimal solutions, but their different move-selection

strategies resulted in differences in both final tour quality and computation time. The experiment therefore illustrates that the choice between best-improvement and first-improvement affects not only the computational procedure, but also the local optimum ultimately reached.

3.6 Part 7: GRASPed Insertion with 2-opt Local Search

Figure 2 compares the objective values obtained before and after applying 2-opt local search to the GRASPed expensive insertion solutions for the `berlin52` instance. A total of 500 outer iterations were performed. Each point represents one outer iteration, where the x-coordinate is the cost of the selected GRASP solution before local search and the y-coordinate is the cost of the corresponding solution after 2-opt.

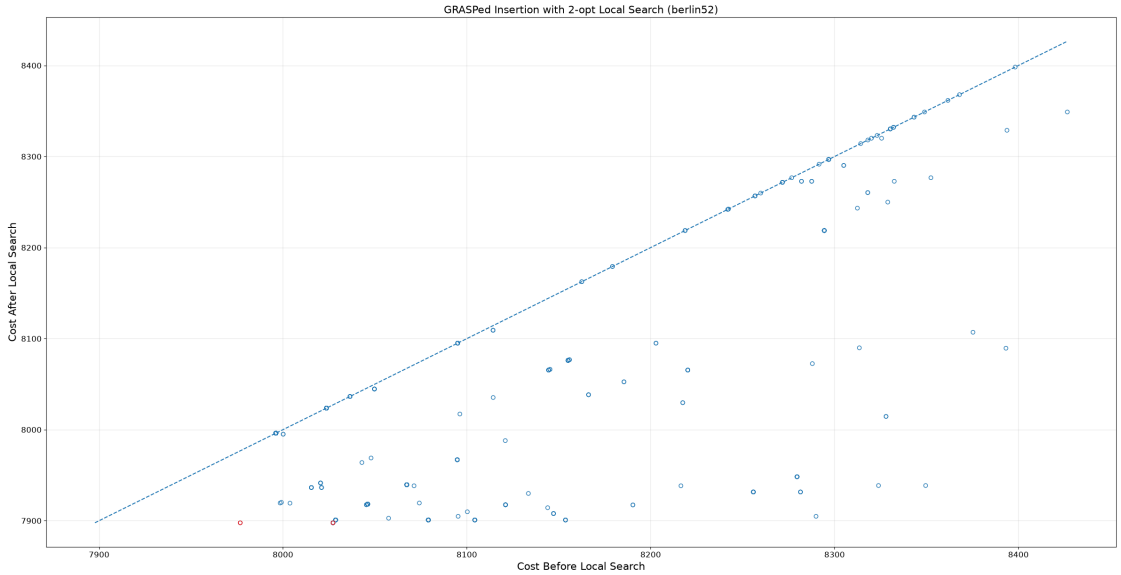


Figure 2: Tour costs before and after applying 2-opt local search to the GRASPed expensive insertion solutions for `berlin52`.

The dashed $y = x$ line represents equal costs before and after local search. All points lie on or below this line, showing that the 2-opt procedure either improves the GRASP solution or leaves its cost unchanged. Points farther below the line indicate larger improvements from the 2-opt procedure.

Some relatively high-cost GRASP solutions are substantially improved by 2-opt, while others remain close to their original cost. Different randomized GRASP constructions can therefore lead the 2-opt search towards different local optima.

The lowest cost obtained by the GRASP construction before local search was 7976.76, while the best solution obtained after 2-opt had a cost of 7897.74, corresponding to an improvement of approximately 0.99%. The best solutions before and after local search are highlighted in red in Figure 2 and originate from different iterations, illustrating that the best GRASP solution does not necessarily lead to the best solution after local search. Finally, the best tour with cost 7897.74 was checked using `isTwoOpt` and `isFeasible`. Both checks returned `True`, confirming that the final solution is feasible and 2-optimal.

4 Conclusion

This assignment investigated constructive, randomized, and local-search heuristics for the TSP. The results showed that Nearest Neighbour is sensitive to its starting city, while GRASPed Expensive Insertion improved upon the deterministic insertion method by exploring alternative construction paths. The 2-opt experiments showed that best-improvement and first-improvement can reach different local optima even when starting from identical tours.

Combining GRASPed insertion with 2-opt further improved the solution on `berlin52`, reducing the best cost from 7976.76 before local search to 7897.74 after local search. A limitation of the experiments is that parameters such as α and the number of GRASP iterations were fixed, and the combined method was mainly evaluated on `berlin52`. Future work could therefore investigate parameter tuning, additional instances, and more efficient or extended local-search methods.

References

[1] R. van Lieshout, "Constructive heuristics," Sept. 2026. Lecture notes for Large-Scale Optimization for Transport & Mobility, Eindhoven University of Technology, Department of Industrial Engineering and Innovation Sciences.

[2] R. van Lieshout, "Local search," Sept. 2026. Lecture notes for Large-Scale Optimization for Transport & Mobility, Eindhoven University of Technology, Department of Industrial Engineering and Innovation Sciences.

Appendix A: AI Assistance for Part 6

The AI assistant used was ChatGPT by OpenAI, using the GPT-5.6 Sol model.

The prompt specified in Appendix A of the assignment was submitted to the AI assistant in a new conversation without any additional context. The initial response generated the method `makeTwoOpt_AI`, which implements a first-improvement 2-opt local search.

No correction rounds were required for the generated algorithm. One follow-up round was performed in which the AI assistant was asked to add comments and descriptions to the generated method. The underlying algorithm and its first-improvement strategy were not changed; the method was only extended with a docstring and explanatory comments. No manual algorithmic changes were made to the generated method.

The following pages contain the complete conversation with the AI assistant.

Today 4:49 PM

I have a Python class representing a symmetric Euclidean Traveling Salesman Problem with the following interface:

```
class TSP:
  self.nCities # int, the number of cities
  self.cities # list of ints, 0 ..  nCities-1
  self.distMatrix # numpy 2D array, distMatrix[i,j] is the distance from city i to city j
  def computeCosts(self, tour) # tour is a list of ints; returns the length of the closed tour
```

Write a method makeTwoOpt_AI(self, tour) that applies 2-exchanges to the given tour until no improving 2-exchange remains, and returns the resulting tour. Return only the method.

```python
def makeTwoOpt_AI(self, tour):
    tour = tour.copy()
    n = len(tour)

    while True:
        improved = False

        for i in range(n):
            for j in range(i + 2, n):
                # Avoid exchanging two adjacent edges through the wrap-around
                if i == 0 and j == n - 1:
                    continue

                a = tour[i]
                b = tour[(i + 1) % n]
                c = tour[j]
                d = tour[(j + 1) % n]

                old_cost = self.distMatrix[a, b] + self.distMatrix[c, d]
                new_cost = self.distMatrix[a, c] + self.distMatrix[b, d]

                if new_cost < old_cost:
                    # Replace edges (a,b), (c,d) with (a,c), (b,d)
                    tour[i + 1:j + 1] = reversed(tour[i + 1:j + 1])
                    improved = True
                    break

            if improved:
                break
```

Python Run

```
        break
```

```
    return tour
```

Convert to PDF

Can you also write comments and descriptions to the method.

Sure. Here is the same method with a proper docstring and comments explaining the 2-opt logic:

```python
def makeTwoOpt_AI(self, tour):
    """
    Improve a given TSP tour using the 2-opt local search heuristic.

    The 2-opt heuristic repeatedly considers pairs of edges in the current
    tour. If removing two edges and reconnecting the tour in the alternative
    way decreases the total tour length, the corresponding section of the
    tour is reversed.

    Parameters
    ----------
    tour : list of int
        Current TSP tour, where each integer represents a city.

    Returns
    -------
    list of int
        A locally improved tour for which no improving 2-opt exchange remains.
    """

    # Make a copy so that the original tour is not modified
    tour = tour.copy()
    n = len(tour)

    # Continue searching until no improving 2-opt move can be found
    while True:
        improved = False

        # Select the first edge (a, b)
        for i in range(n):
```

Python Run

```
                # Select the second edge (c, d).
                # Starting at i + 2 ensures that the two selected edges
                # are not adjacent.
                for j in range(i + 2, n):

                    # When i = 0 and j = n - 1, the two edges are also
                    # adjacent because the tour is closed.
                    if i == 0 and j == n - 1:
                        continue

                    # Current edges:
                    # (a, b) = (tour[i], tour[i+1])
                    # (c, d) = (tour[j], tour[j+1])
                    a = tour[i]
                    b = tour[(i + 1) % n]
                    c = tour[j]
                    d = tour[(j + 1) % n]

                    # Cost of the two edges currently in the tour
                    old_cost = (
                        self.distMatrix[a, b]
                        + self.distMatrix[c, d]
                    )

                    # Cost after the proposed 2-opt exchange:
                    # (a,b) and (c,d) are replaced by (a,c) and (b,d)
                    new_cost = (
                        self.distMatrix[a, c]
                        + self.distMatrix[b, d]
                    )

                    # Perform the exchange only if it improves the tour
                    if new_cost < old_cost:

                        # Reversing this section reconnects the tour using
                        # the new edges (a,c) and (b,d)
                        tour[i + 1:j + 1] = reversed(tour[i + 1:j + 1])

                        improved = True
                        break

                # Restart the search from the beginning after an improvement
                if improved:
                    break

            # If a complete search found no improvement, the tour is
            # 2-opt locally optimal
            if not improved:
                break

        return tour
```

This implementation uses **first improvement**: as soon as it finds an improving 2-opt exchange, it applies it and restarts the search. It terminates only after checking all valid edge pairs without finding an improvement.

Convert to PDF

ChatGPT can make mistakes. Check important info